Importing Libraries

```
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import pandas as pd
sns.set()
import numpy as np

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

```

```
df = pd.read_csv('heart.csv')
df.head()
```

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa
0	63	1	3	145	233	1	0	150	0	2.3	0	0
1	37	1	2	130	250	0	1	187	0	3.5	0	0
2	41	0	1	130	204	0	0	172	0	1.4	2	0
3	56	1	1	120	236	0	1	178	0	0.8	2	0
4	57	0	0	120	354	0	1	163	1	0.6	2	0

Exploratory Data Analysis

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trtbps    303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalachh  303 non-null    int64
 8   exng      303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slp       303 non-null    int64
 11  caa       303 non-null    int64
 12  thall     303 non-null    int64
 13  output    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

```
df.isnull()
```

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak
0	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...
298	False	False	False	False	False	False	False	False	False	False
299	False	False	False	False	False	False	False	False	False	False
300	False	False	False	False	False	False	False	False	False	False
301	False	False	False	False	False	False	False	False	False	False
302	False	False	False	False	False	False	False	False	False	False

303 rows × 14 columns

```
#Checking if there is null value
df.isnull().sum()
```

```
age         0
sex         0
cp          0
trtbps      0
chol        0
fbs         0
restecg     0
thalachh    0
exng        0
oldpeak     0
slp         0
caa         0
thall       0
output      0
dtype: int64
```

```
#Checking if there is zero value

#replace 0 value with NaN
df_copy = df.copy(deep = True)
df_copy[['trtbps','chol','thalachh','oldpeak']] = df_copy[['trtbps','chol','thala

# Showing the Count of NANs
print(df_copy.isnull().sum())
```

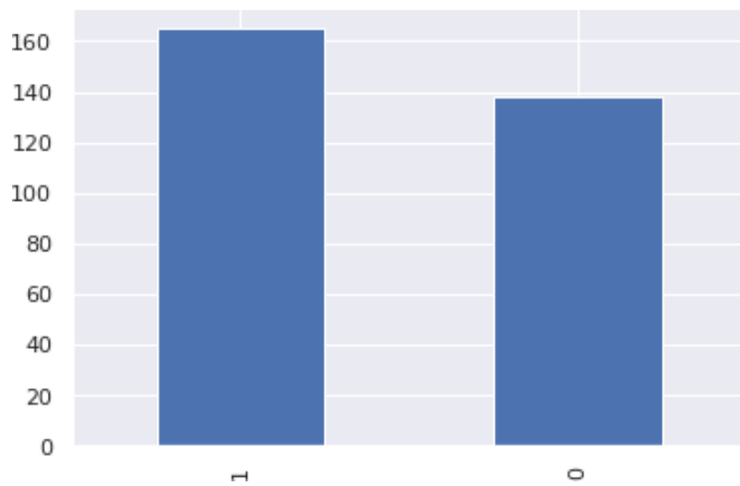
```
age          0
sex          0
cp           0
trtbps       0
chol         0
fbs          0
restecg      0
thalachh     0
exng         0
```

```
oldpeak     99
slp          0
caa          0
thall        0
output       0
dtype: int64
```

```
#Fill null value with median
df_copy['oldpeak'].fillna(df_copy['oldpeak'].mean(), inplace = True)
```

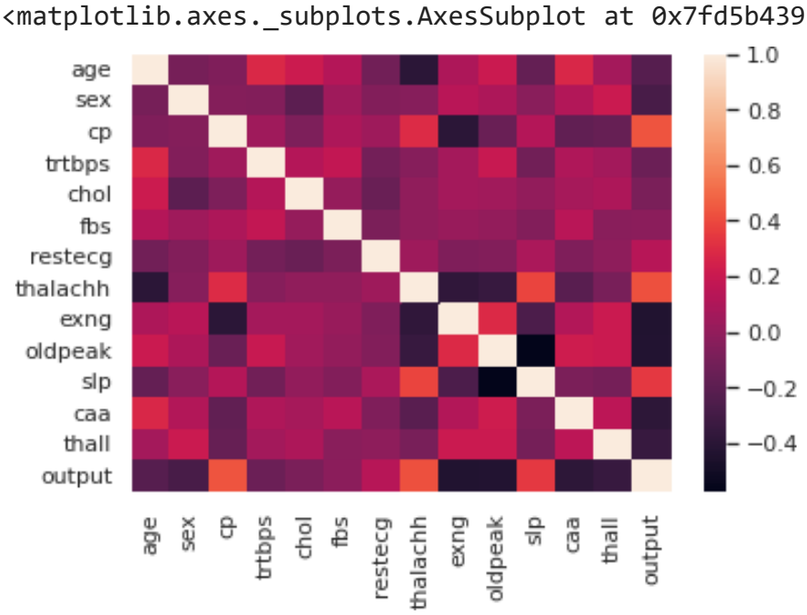
```
#Counting 1 and 0 Value in Output column
color_wheel = {1: "#0392cf", 2: "#7bc043"}
colors = df["output"].map(lambda x: color_wheel.get(x + 1))
print(df.output.value_counts())
p=df.output.value_counts().plot(kind="bar")
```

```
1    165
0    138
Name: output, dtype: int64
```




```
#See the correlation between each column
sns.heatmap(df.corr())
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fd5b4391a10>
```



Machine Learning Model Building

```
X = df.drop('output', axis=1)
```

```
y = df['output']
```

```
#test size 20% and train size 80%
from sklearn.model_selection import train_test_split, cross_val_score, cross_val_
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2,random_sta
```

Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

dtree = DecisionTreeClassifier()
dtree.fit(X_train, y_train)
```

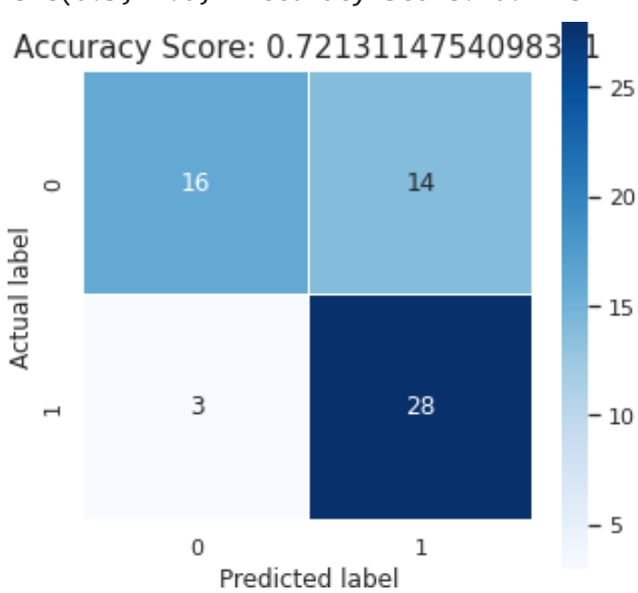
```
DecisionTreeClassifier()
```

```
y_pred = dtree.predict(X_test)
print("Classification report - \n", classification_report(y_test,y_pred))
```

```
Classification report -
               precision    recall  f1-score   support

           0       0.84      0.53      0.65        30
           1       0.67      0.90      0.77        31

    accuracy                           0.72        61
   macro avg       0.75      0.72      0.71        61
weighted avg       0.75      0.72      0.71        61
```

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,5))
sns.heatmap(data=cm,linewidths=.5, annot=True,square = True,  cmap = 'Blues')
plt.ylabel('Actual label')
plt.xlabel('Predicted label')
all_sample_title = 'Accuracy Score: {0}'.format(dtree.score(X_test, y_test))
plt.title(all_sample_title, size = 15)
```

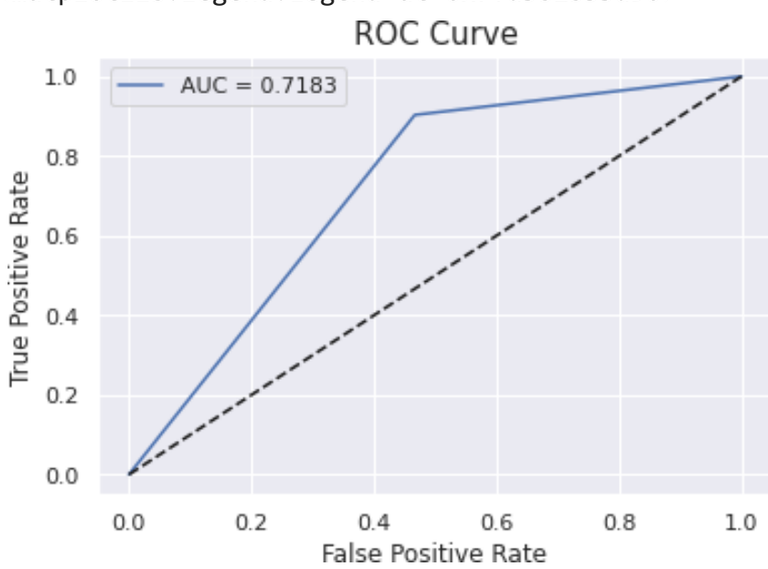
```
Text(0.5, 1.0, 'Accuracy Score: 0.7213114754098361')
```




```
from sklearn.metrics import roc_curve, roc_auc_score
```

```
y_pred_proba = dtree.predict_proba(X_test)[:][:,1]

df_actual_predicted = pd.concat([pd.DataFrame(np.array(y_test), columns=['y_actua
df_actual_predicted.index = y_test.index

fpr, tpr, tr = roc_curve(df_actual_predicted['y_actual'], df_actual_predicted['y_
auc = roc_auc_score(df_actual_predicted['y_actual'], df_actual_predicted['y_pred_

plt.plot(fpr, tpr, label='AUC = %0.4f' %auc)
plt.plot(fpr, fpr, linestyle = '--', color='k')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve', size = 15)
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7fd5b2b35d90>
```



Support Vector Machines

```
from sklearn import svm

svm = svm.SVC(kernel='linear', gamma='auto', probability=True)
svm.fit(X_train, y_train)
```

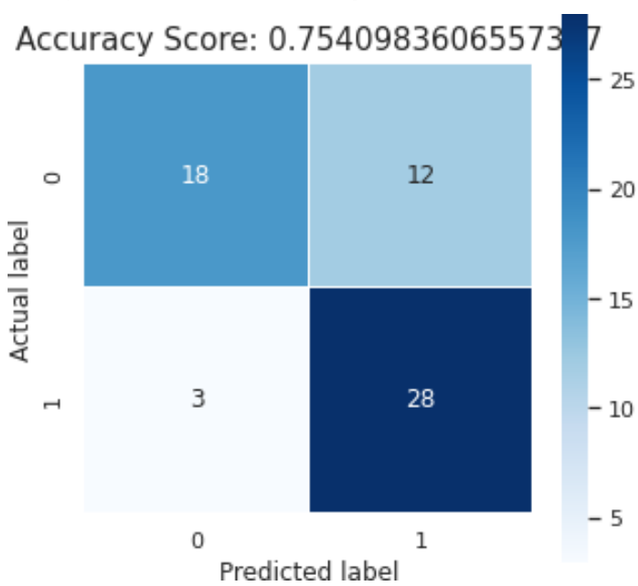
```
SVC(gamma='auto', kernel='linear', probability=True)
```

```
y_pred = svm.predict(X_test)
print("Classification report - \n", classification_report(y_test,y_pred))
```

```
Classification report -
               precision    recall  f1-score   support

           0       0.86      0.60      0.71        30
           1       0.70      0.90      0.79        31

    accuracy                           0.75        61
   macro avg       0.78      0.75      0.75        61
weighted avg       0.78      0.75      0.75        61
```

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,5))
sns.heatmap(data=cm,linewidths=.5, annot=True,square = True,  cmap = 'Blues')
plt.ylabel('Actual label')
plt.xlabel('Predicted label')
all_sample_title = 'Accuracy Score: {0}'.format(svm.score(X_test, y_test))
plt.title(all_sample_title, size = 15)
```

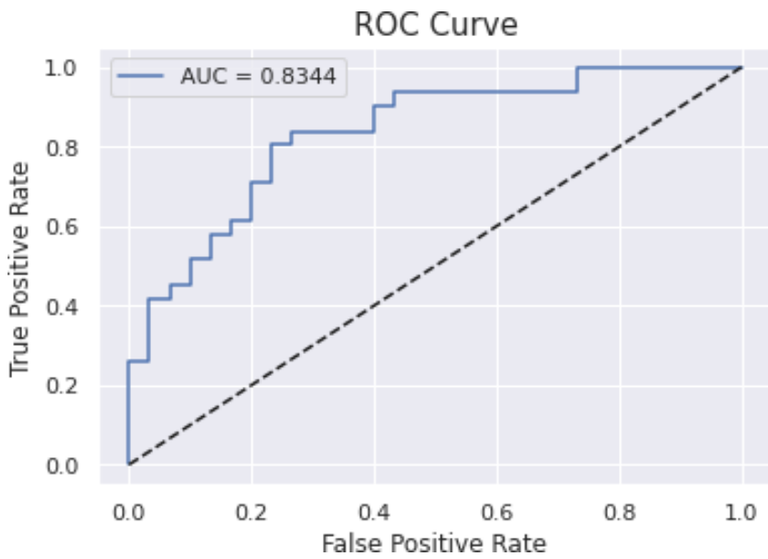
```
Text(0.5, 1.0, 'Accuracy Score: 0.7540983606557377')
```




```
from sklearn.metrics import roc_curve, roc_auc_score
y_pred_proba = svm.predict_proba(X_test)[:][:,1]

df_actual_predicted = pd.concat([pd.DataFrame(np.array(y_test), columns=['y_actua
df_actual_predicted.index = y_test.index

fpr, tpr, tr = roc_curve(df_actual_predicted['y_actual'], df_actual_predicted['y_
auc = roc_auc_score(df_actual_predicted['y_actual'], df_actual_predicted['y_pred_

plt.plot(fpr, tpr, label='AUC = %0.4f' %auc)
plt.plot(fpr, fpr, linestyle = '--', color='k')
```

```
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve', size = 15)
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7fd5b1a08c50>
```



Logistic Regression

```
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression()
logreg.fit(X_train, y_train)
```

```
/usr/local/lib/python3.7/dist-packages/sklearn/linear_model/_logistic.py:818:
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regress
  extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG,
LogisticRegression()
```

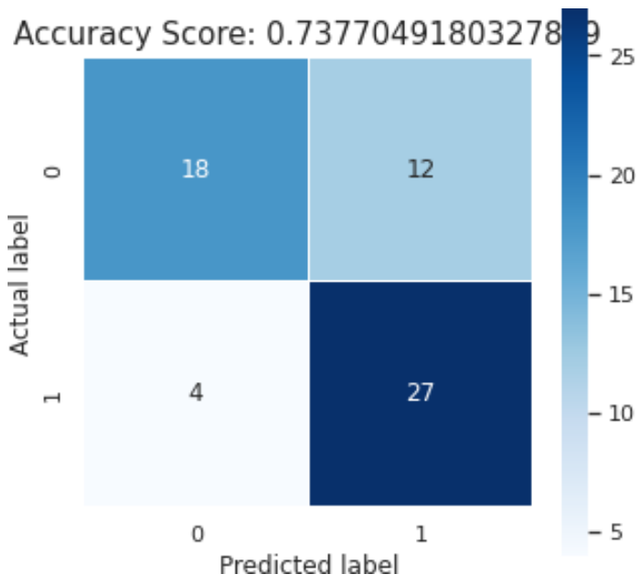
```
y_pred = logreg.predict(X_test)
print("Classification report - \n", classification_report(y_test,y_pred))
```

```
Classification report -
               precision    recall  f1-score   support

           0       0.82      0.60      0.69        30
           1       0.69      0.87      0.77        31

    accuracy                           0.74        61
   macro avg       0.76      0.74      0.73        61
weighted avg       0.75      0.74      0.73        61
```

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,5))
sns.heatmap(data=cm,linewidths=.5, annot=True,square = True,  cmap = 'Blues')
```

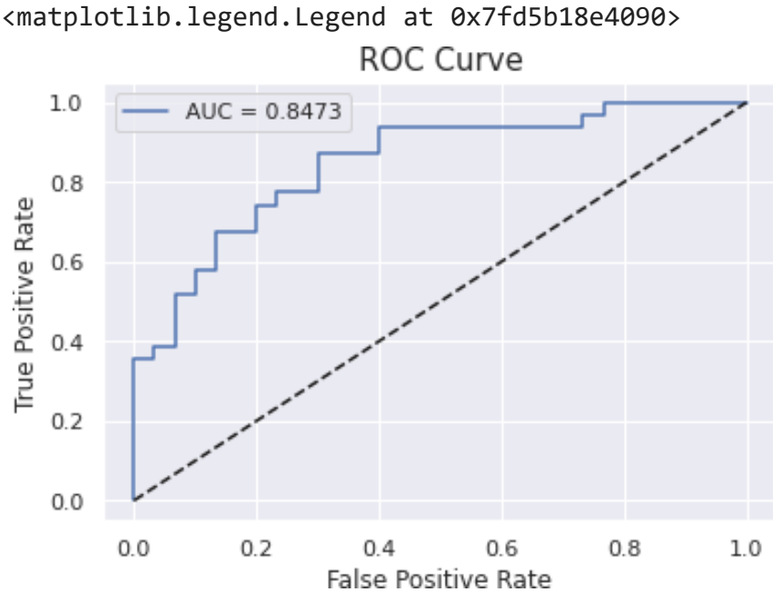
```
plt.ylabel('Actual label')
plt.xlabel('Predicted label')
all_sample_title = 'Accuracy Score: {0}'.format(logreg.score(X_test, y_test))
plt.title(all_sample_title, size = 15)
```

```
Text(0.5, 1.0, 'Accuracy Score: 0.7377049180327869')
```




```
from sklearn.metrics import roc_curve, roc_auc_score
y_pred_proba = logreg.predict_proba(X_test)[:][:,1]

df_actual_predicted = pd.concat([pd.DataFrame(np.array(y_test), columns=['y_actua
df_actual_predicted.index = y_test.index

fpr, tpr, tr = roc_curve(df_actual_predicted['y_actual'], df_actual_predicted['y_
auc = roc_auc_score(df_actual_predicted['y_actual'], df_actual_predicted['y_pred_

plt.plot(fpr, tpr, label='AUC = %0.4f' %auc)
plt.plot(fpr, fpr, linestyle = '--', color='k')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve', size = 15)
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7fd5b18e4090>
```



K Nearest Neighbors

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
```

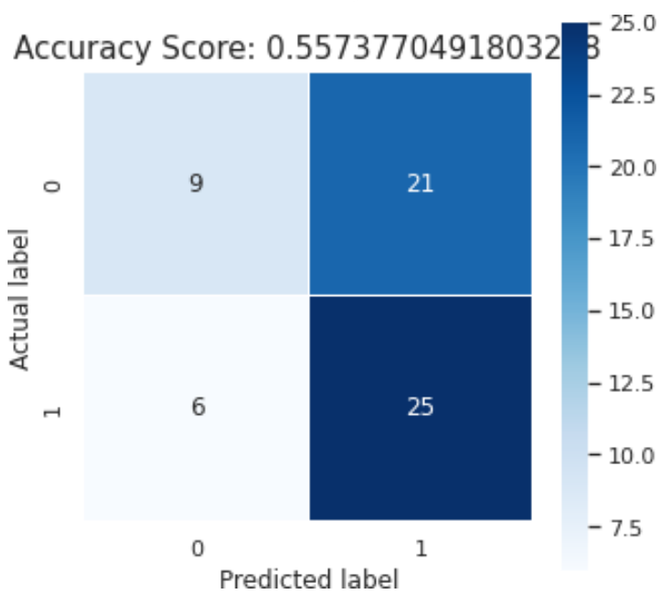
```
KNeighborsClassifier()
```

```
y_pred = knn.predict(X_test)
print("Classification report - \n", classification_report(y_test,y_pred))
```

```
Classification report -
               precision    recall  f1-score   support

           0       0.60      0.30      0.40        30
           1       0.54      0.81      0.65        31

    accuracy                           0.56        61
   macro avg       0.57      0.55      0.52        61
weighted avg       0.57      0.56      0.53        61
```

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,5))
sns.heatmap(data=cm,linewidths=.5, annot=True,square = True,  cmap = 'Blues')
plt.ylabel('Actual label')
plt.xlabel('Predicted label')
all_sample_title = 'Accuracy Score: {0}'.format(knn.score(X_test, y_test))
plt.title(all_sample_title, size = 15)
```

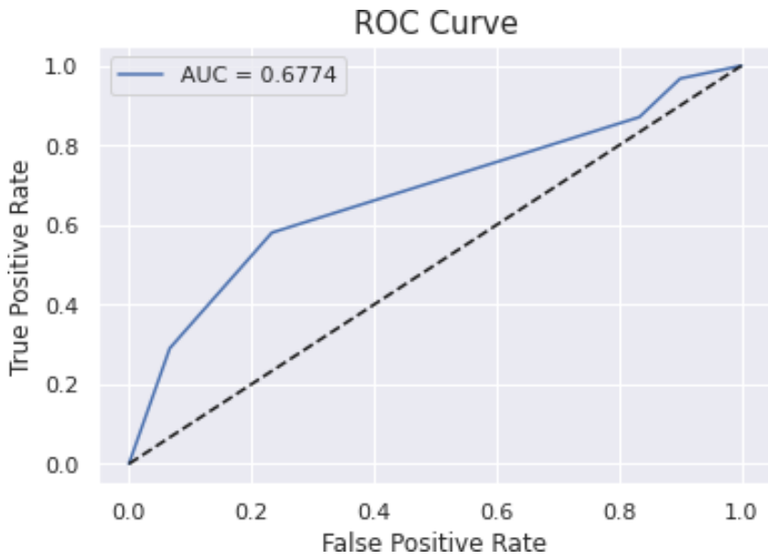
```
Text(0.5, 1.0, 'Accuracy Score: 0.5573770491803278')
```




```
from sklearn.metrics import roc_curve, roc_auc_score
y_pred_proba = knn.predict_proba(X_test)[:][:,1]

df_actual_predicted = pd.concat([pd.DataFrame(np.array(y_test), columns=['y_actua
df_actual_predicted.index = y_test.index

fpr, tpr, tr = roc_curve(df_actual_predicted['y_actual'], df_actual_predicted['y_
auc = roc_auc_score(df_actual_predicted['y_actual'], df_actual_predicted['y_pred_

plt.plot(fpr, tpr, label='AUC = %0.4f' %auc)
plt.plot(fpr, fpr, linestyle = '--', color='k')
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
plt.title('ROC Curve', size = 15)
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7fd5b16fe2d0>
```



Colab paid products - Cancel contracts here0s

completed at 1:03 PM